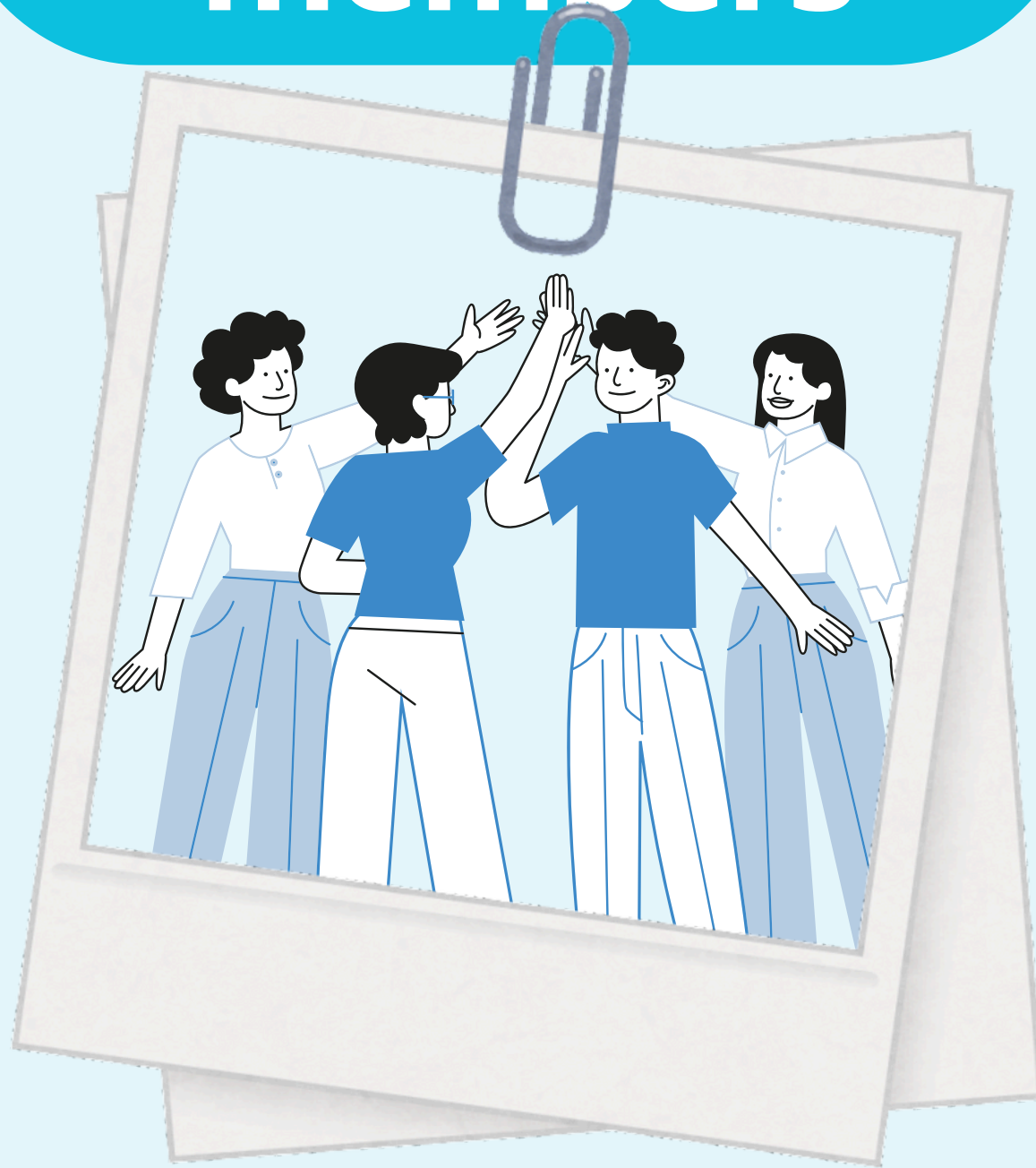


PASSENGER AIRLINE

Trabalho realizado por:
Beatriz Martins
Laura Barreto



Team members



Beatriz Martins

➤ uc2023210538@student.uc.pt

➤ 962257282

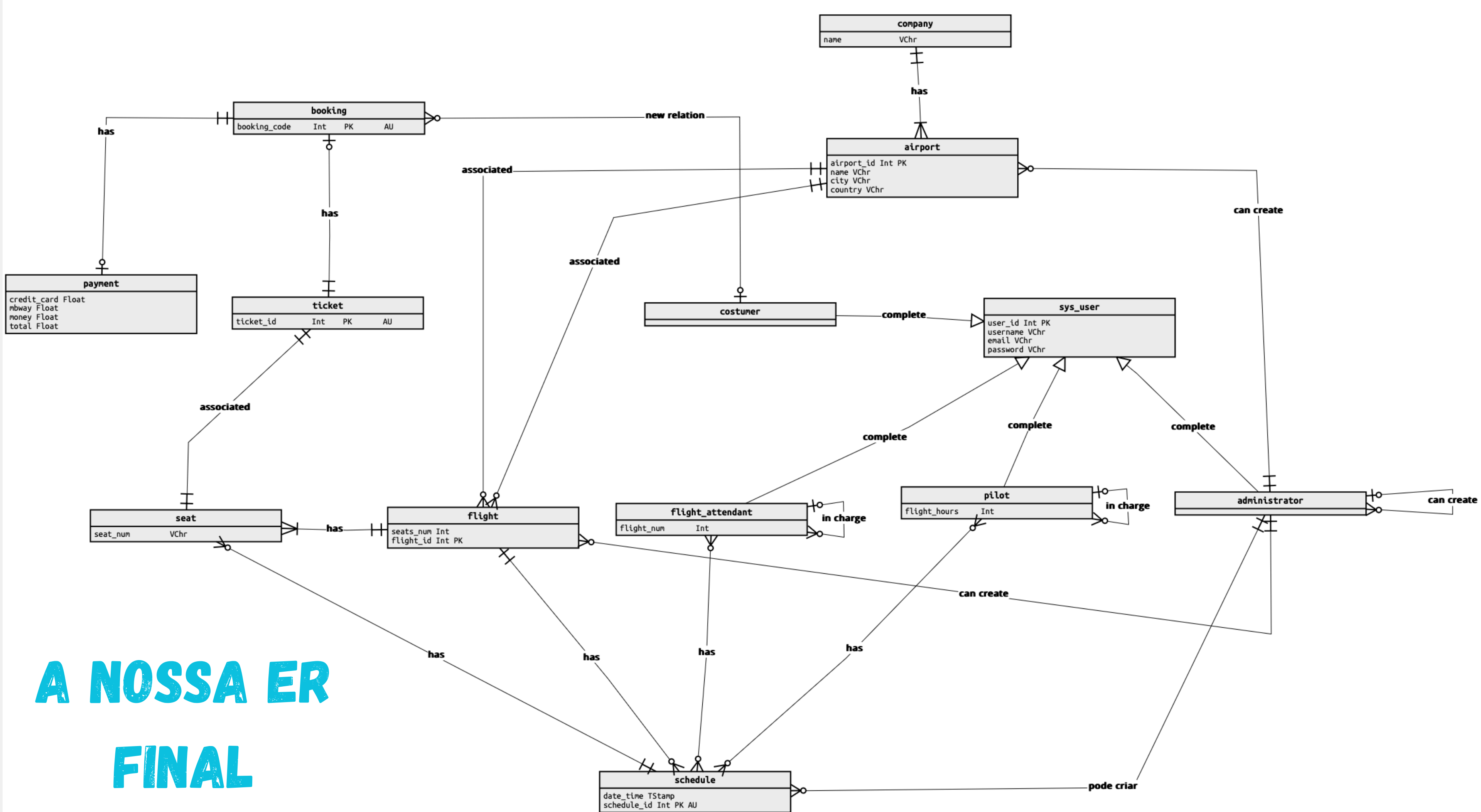
Laura Barreto

➤ uc2023214375@student.uc.pt

➤ 925984986

Suposições

- ✿ Um copiloto pode ser considerado um piloto (e pode desempenhar as suas funções depois de 150 horas de voo);
- ✿ Membros da tripulação apenas podem supervisionar outros membros com a mesma função;
- ✿ Um avião tem vários assentos cada um com um formato: número + letra (a letra representa o nº da fila e o número representa o nº da linha do respetivo assento);
- ✿ Pode haver mais do que um aeroporto numa mesma cidade;
- ✿ Um voo está associado a dois aeroportos (um de origem e um de chegada)
- ✿ Se um voo tiver o mesmo destino e origem no mesmo dia que outro, então terão ids diferentes;
- ✿ Só os administradores podem criar aeroportos, voos, e horários que são instâncias de um voo.
- ✿ Todos os utilizadores podem comprar bilhetes sejam eles clientes ou pessoas que trabalham na companhia;
- ✿ Assumimos que uma pessoa, quando faz a reserva de um bilhete, já fica pago e que esta irá estar presente no voo.
- ✿ Assumimos que quando um horário é criado com um id de voo então os assentos desse voo nesse horário são todos criados automaticamente tal como os bilhetes desse horário nesse avião.



A NOSSA ER
FINAL

Estrutura do software

- Trabalhámos com um sistema de API REST para uma companhia aérea, que interage com uma base de dados para gerir voos, utilizadores, reservas, horários entre outros. Utilizamos linguagem sql e python e criámos um ambiente isolado onde importámos os módulos necessários (instruções no relatório).

1. Medidas de Segurança

A autenticação é feita utilizando JWT, e apenas utilizadores autenticados podem aceder aos endpoints restritos. As palavras-passe são armazenadas de forma segura através de hashing antes de serem guardadas na base de dados.

2. Validação de Dados e Gestão de Erros

Os dados de entrada são validados para garantir a sua correção, e os erros são tratados de forma a fornecer mensagens e códigos de erro claros.



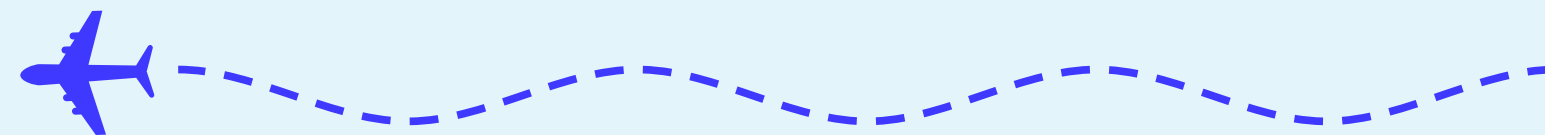
Transações e Concorrência

Transações:

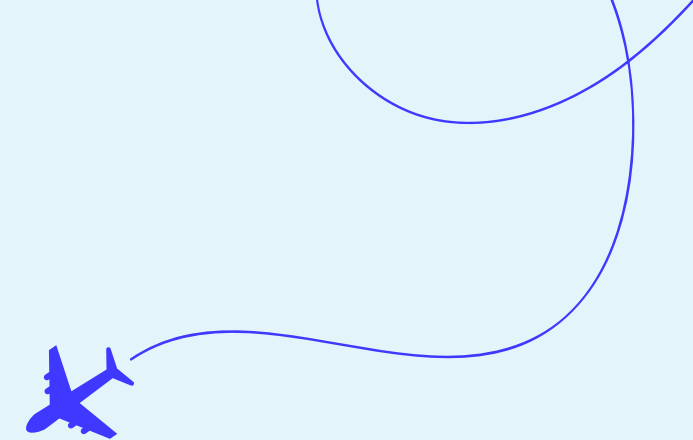
- Usar comandos como “COMMIT” e “ROLLBACK” no SQL permite a garantia de atomicidade e consistência. Ao usar `conn.commit()` e `conn.rollback()`, podemos confirmar ou reverter alterações.

Concorrência:

- Quando alguma transação está a ser executada como, por exemplo, se um cliente quiser reservar algum voo num determinado horário com um determinado assento, então esse assento fica bloqueado, não permitindo a outros clientes de realizarem a mesma transação. Por outras palavras, o nosso software não permite situações de concorrência, prevenindo situações de descontentamento e proporcionando uma experiência de usuário confiável.



Falhas e erros

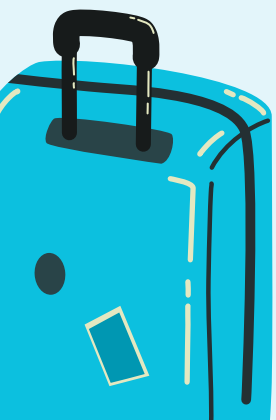


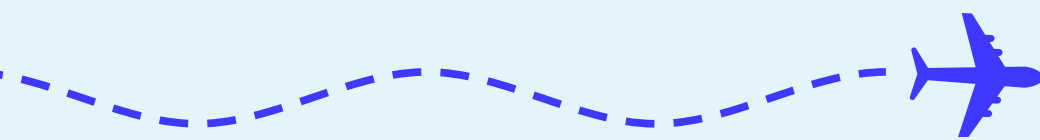
Falhas:

- Se houver alguma falha como, por exemplo, haver uma falha na rede, o ideal seria que o sistema continuasse a funcionar e não perdesse as transações realizadas até este ponto, no entanto caso isto aconteça, também é importante enviar mensagens de erro para o utilizador a informá-lo do acontecido, caso o sistema deixasse de funcionar.

Erros:

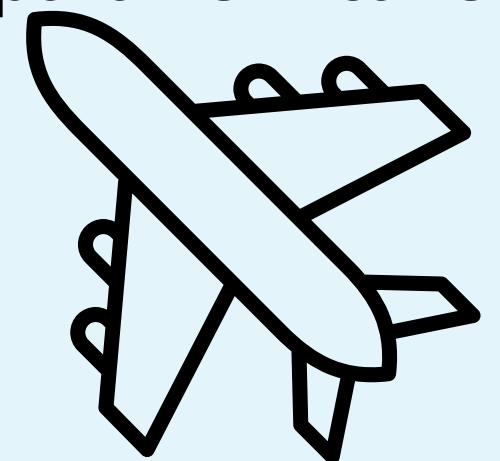
- Caso não seja possível realizar uma transação devido a alguma falha, é importante avisar o utilizador do sucedido e utilizamos “log” para passar uma mensagem do erro com orientações ao utilizador.
- É então essencial fornecer mensagens de erro e códigos de status adequados nas respostas da API, como iremos usar 200 para sucesso da transação requerida, 400 para pedidos inválidos e 500 para erros do servidor.





Testes e qualidade

- O processo de testes e avaliação de qualidade no desenvolvimento de software é essencial para garantir a entrega de um sistema robusto e funcional, que atenda aos requisitos dos usuários e não possua falhas críticas.
- Desta forma, experimentámos as funções que criámos para verificar se não aconteciam situações que não pretendíamos obter. Como por exemplo, ser possível reservar um bilhete que já estivesse reservado, ou se o nosso sistema permitisse a um utilizador, seja ele qual for, criar os aeroportos ou voos ou ainda, por exemplo, se um utilizador pudesse realizar login sem antes ter feito o registo entre muitas outras situações que não podemos permitir que aconteçam.
- Portanto, de forma a nos certificarmos que não aconteceriam situações inesperadas, executamos as funções e simulamos uma companhia aérea com todas as suas componentes, para verificar se não poderiam surgir anomalias ou situações não esperadas.



1. Register User (POST)

- Permite registrar os vários utilizadores na plataforma criando uma conta com as informações do user_id, username, e-mail e palavra-passe. Qualquer pessoa se pode registrar, no entanto, só administradores (com token válido de administrador) poderão registrar outros administradores.

Verificações:

- Se o payload tem todos os parâmetros necessários para realizar o registo;
- Se o user é um administrador, piloto, hospedeiro ou costumer;
- Se um administrador está a ser registado por outro administrador.



req	POST http://localhost:8080/sgdproj/user { "username": username, "email": email, "password": password, (...) }
res	{ "status": status_code, "errors": errors (if any occurs), "results": user_id (if it succeeds) }

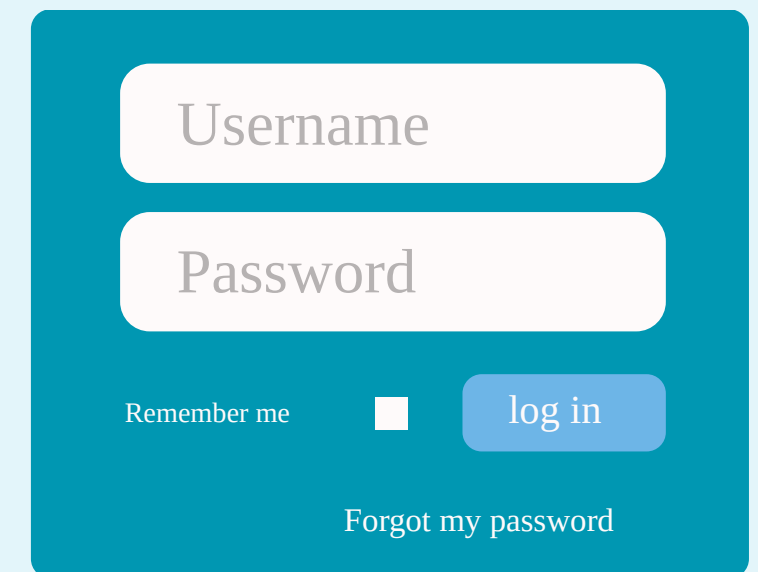
2. Login User (PUT)

- Permite o login de utilizadores na plataforma, autenticando-os através do nome de utilizador e palavra-passe. Se a autenticação for bem-sucedida, o utilizador recebe um token de autenticação (geralmente um JSON Web Token (JWT)) que irá aparecer na saída do postman.

Verificações:

- Se são inseridos os parâmetros username e password no payload;
- Se o utilizador já tem conta registada com os parâmetros dados;
- Se o user_name e a password são iguais aos da conta registada.

Precauções: Proteger a palavra passe!

A login form with a teal background. It features two white input fields for 'Username' and 'Password'. Below the password field is a 'Remember me' checkbox and a blue 'log in' button. At the bottom, there is a link that says 'Forgot my password'.

req	PUT	http://localhost:8080/sgdproj/user
		<code>{"username": username, "password": password}</code>
res		<code>{"status": status_code, "errors": errors (if any occurs), "results": auth_token (if it succeeds) }</code>

3. Add Airport (POST)

- Permite, apenas aos administradores, registrar aeroportos na base de dados com um id, nome, cidade e país.

Verificações:

- Se os parâmetros foram todos inseridos corretamente;
- Se foi um administrador a realizar esta transação;
- Se o id do aeroporto ainda não existe na base de dados.



```
req    POST    http://localhost:8080/sgdproj/airport
        {"airport_code": code, "name": name, "city": city, "country": country, (...)}
res    {"status": status_code, "errors": errors (if any occurs), "results": airport_code (if it
        succeeds) }
```

4. Create Flight (POST)

- Este endpoint também só poderá ser realizado por um administrador, que criará um voo com um aeroporto de origem e um aeroporto de destino e com o número de assentos disponíveis para os passageiros.

Verificações:

- Se os parâmetros foram todos inseridos corretamente;
- Se foi um administrador a realizar esta transação;
- Se os ids de aeroporto de origem e destino existem;
- Se já existe um voo com o mesmo id nesse mesmo dia.



Suposição: Não iremos usar os parâmetros `departure_time` e `landing_time` visto que não apareceram no enunciado e a nossa ideia é que um voo pode ocorrer em muitas horas, consoante os horários a que estiver ligado e não a uma hora única.

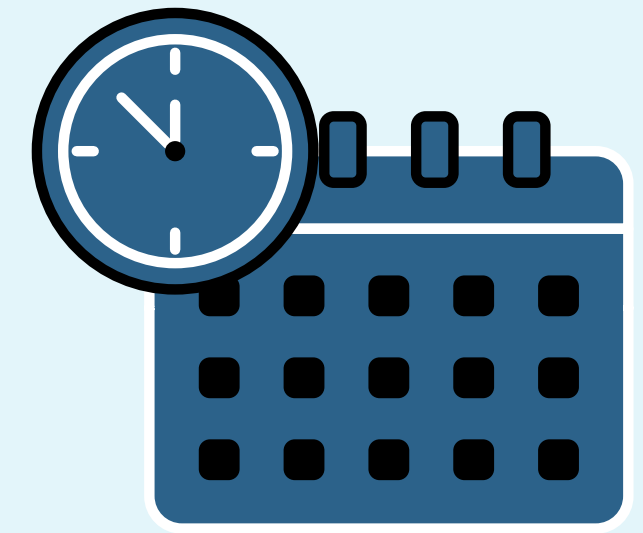
req	POST http://localhost:8080/sgdproj/flight { <code>"flight_code": code, "origin_airport": origin, "destination_airport": destination, "departure_time": time, "landing_time": time, "available_seats": number (...)</code> }
res	{ <code>"status": status_code, "errors": errors (if any occurs), "results": flight_code (if it succeeds)</code> }

5. CREATE SCHEDULE (POST)

- Neste endpoint apenas o administrador poderá criar um horário para um determinado voo, sendo possível haver vários horários de um voo mas não no mesmo dia.

Verificações:

- Se foram inseridos todos os parâmetros necessários e se o id do voo existe;
- Se está a ser criado por um administrador;
- Se o voo associado existe;
- Se o id do voo não está já associado a um horário no mesmo dia



Suposição: Quando um horário é criado são também criados os assentos nesse voo automaticamente.

req	POST http://localhost:8080/sgdproj/schedule { "flight_code": code, "date": date, (...) }
res	{ "status": status_code, "errors": errors (if any occurs), "results": schedule_id (if it succeeds) }

6. Check available routes (GET)

- Qualquer cliente autenticado pode consultar todas as rotas disponíveis, não fornecendo qualquer parâmetro neste caso, ou pode optar por informar um aeroporto de origem, um aeroporto de destino ou ambos, para obter informações mais específicas como os horários de uma determinada rota, se assim o desejar, selecionando a origem e o destino que pretende.

Verificações:

- Podemos verificar se o payload tem os parâmetros corretos caso o utilizador escolha meter algum voo origem ou destino, ambos ou nenhum.

```
req  GET  http://localhost:8080/sgdproj/check_routes
      {"origin_airport": code, "destination_airport": code, (...)}

res  {"status": status_code, "errors": errors (if any occurs), "results": [{"origin_airport":
code, "destination_airport": code, "flight_code": code, "schedules": [...]}, ...
{"origin_airport": code, "destination_airport": code, "flight_code": code, "schedules":
[...]}} (if it succeeds)}
```



7. Check available seats (GET)

- Qualquer utilizador pode obter os assentos disponíveis num determinado voo a um determinado horário.

Verificações:

- Se o payload tem todos os parâmetros necessários
- Se o id do voo no request existe
- Se o id do horário no request existe
- Se existe um determinado horário com o id do voo em questão



req	GET	http://localhost:8080/sgdproj/check_seats { "flight_code": code, "schedule_id": id, (...) }
res		{ "status": status_code, "errors": errors (if any occurs), "results": ["1A", "1B", "1C" ..., "32F", "32G", "32H"] }



8. Book Flight (POST)



- Todos os utilizadores podem fazer a reserva sendo que, quando a fazem, estão na verdade a reservar um bilhete com um determinado assento, num determinado voo, a uma determinada hora.

Verificações:

- Se os parâmetros estão corretos e existem na base de dados;
- Se o assento do bilhete que queremos reservar não está já reservado (evitando concorrências);
- Se o bilhete do voo que queremos reservar num certo horário existe.

req	POST http://localhost:8080/sgdproj/book_flight { "flight_code": code, "schedule_id": id, "seat_id": id (...) }
res	{ "status": status_code, "errors": errors (if any occurs), "results": schedule_id (if it succeeds) }



9. Generate a report with top N destinations (GET)

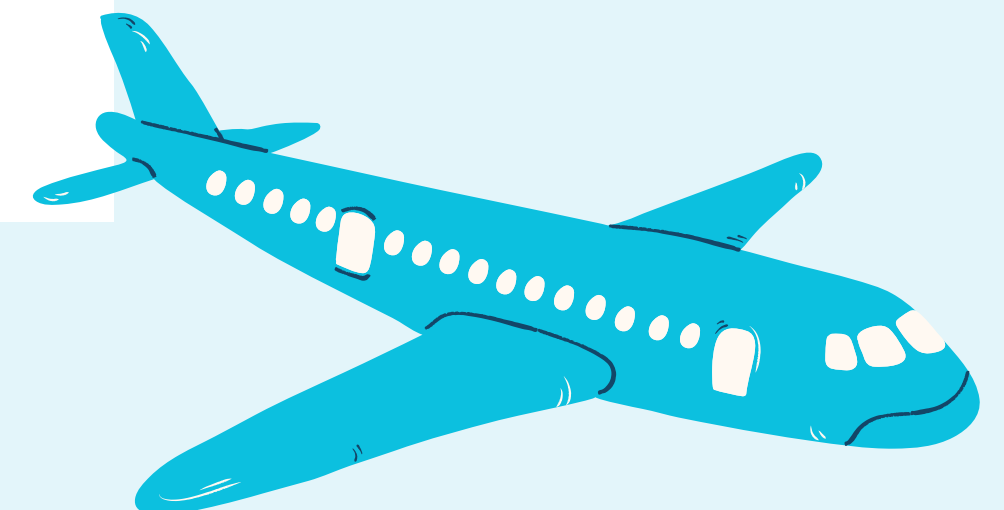
- O objetivo é gerar um relatório dos N destinos com mais voos nos últimos 12 meses. Portanto, a consulta deve obter como resultado, os N destinos mais recorrentes, ou seja, os destinos que a mais voos estão associados. Estes são ordenados por ordem decrescente, tendo como critério a quantidade de voos associados a esse destino.

Verificações:

- Se o valor de N é maior que zero.

```
req    GET    http://localhost:8080/sgdproj/report/topDestinations/{N}

res    {"status": status_code, "errors": errors (if any occurs), "results": [
        {"destination_airport": code, "number_flights": number},
        {"destination_airport": code, "number_flights": number},
        ...,
        {"destination_airport": code, "number_flights": number},
    ] (if it succeeds)}
```



10. Generate a (monthly) report with the top N routes with more passengers (GET)

- O objetivo é gerar um relatório dos N destinos com mais reservas, em cada mês, nos últimos 12 meses. Portanto, a consulta deve obter os N voos mais populares entre os passageiros, em cada mês do ano. Estes vêm ordenados por ordem decrescente, tendo como critério, o total de passageiros.

Verificações:

- Se o valor de N tem que ser maior que zero

Suposições: O parâmetro N tem de ser passado como parâmetro mesmo que não esteja presente no caminho do enunciado. Supomos também, que se uma pessoa fizer uma reserva de um bilhete, então irá no voo.

```
req    GET    http://localhost:8080/sgdproj/report/topRoutes

res    {"status": status_code, "errors": errors (if any occurs), "results": [
        {"month": "month_0_year",
         "topN": [{"flight_id": ..., "total_passengers": ...}, {"flight_id": ...,
         "total_passengers": ..} ..., {"flight_id": ..., "total_passengers": ...}] },
        {"month": "month_1_year",
         "topN": [{"flight_id": ..., "total_passengers": ...}, {"flight_id": ...,
         "total_passengers": ..} ..., {"flight_id": ..., "total_passengers": ...}] },
        ...
        {"month": "month_11_year",
         "topN": [{"flight_id": ..., "total_passengers": ...}, {"flight_id": ...,
         "total_passengers": ..} ..., {"flight_id": ..., "total_passengers": ...}] }
      ] (if it succeeds)}
```





FIM

Trabalho feito em conjunto:

Beatriz Martins: 60 horas

Laura Barreto: 60 horas